

Aryan Sharma

Riverside, CA

aryan250403@gmail.com

1-951-830-5938

[LinkedIn](#)

[GitHub](#)

[Google Scholar](#)

[Website](#)

EDUCATION

Master of Science in Computer Science, University of California Riverside, CA (GPA: 3.7/4)

Sep 2025 – Present

Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Vellore, India (GPA: 3.9/4)

Sep 2021 – Jun 2025

WORK EXPERIENCE

Cybersecurity and Data analysis Intern, Redinent Innovations, Bengaluru

Sep 2023 – Dec 2023

- Aggregated banner fingerprints from exposed interfaces of IOT devices using dorks, Censys, Shodan, and ZoomEye, expanded the database by over 1200+ entries, achieving 25% growth.
- Pinpointed 17 vertical parameters to uniquely identify login pages for specific subgroups of devices.
- Compiled stable attributes such as tuples through recursive analysis of login pages and reduced false matches by 25%, improving identification accuracy to 91%.
- Delivered a clean, usable dataset mapped to device identifiers and helped streamline downstream tasks like external attack surface monitoring, increasing the number of devices monitored by 35%.

3D Modeling, Animations, and Game Development Intern, Viftr Technologies, Vellore

Feb 2023 – May 2023

- Architected and refined 25+ environmental, character, and animated assets every week, improving the assets library by 275%.
- Spearheaded 5-person team to deliver 10 core features, completing the project 3 weeks before schedule.
- Pioneered shader-based solutions in Unity to replace simulation-based methods for environmental objects; enhanced runtime efficiency by 60% and reduced draw calls by 150 per frame.
- Engineered pipelines integrating accessibility features within the game, resulting in a 65% surge in user engagement among children with neurodevelopmental disorders during initial user testing.

TECHNICAL SKILLS

Programming Languages:

Python (Advanced), C++ (Intermediate), Java (Intermediate)

Technical Expertise:

ML, DL, Data Science, Natural Language Processing, Computer Vision, GenAI, Retrieval Augmented Generation, Ultralytics YOLOv8

Tools And Libraries:

Cohere, Qdrant, tflite_flutter, LangChain, Pandas, Numpy, Matplotlib, BeautifulSoup, Pytorch, Transformers, ONNX, TFLite, FastAPI

Databases:

SQL, Riverpod, Isar (NoSQL), DuckDB

Front-end Tools:

HTML, CSS, JavaScript, TypeScript, React, Vite, Dart

PUBLICATIONS

A cross dataset meta-model for hepatitis C detection using multi-dimensional pre-clustering

Mar 2025 (Published)

Aryan Sharma, Tanmay Khade & Shashank Mouli Satapathy

Scientific Reports (5-year IF: 4.3 (2024))

PROJECTS

FinRAG: Agentic RAG over SEC 10-k filings | *Python, TypeScript, Cohere, Qdrant, DuckDB, FastAPI, Next.js* · Active development; agent + evaluation layers in progress

May 2026 - Present

- Designed and shipped a 3-stage retrieval funnel (BM25 + Cohere embed-v3 fused via RRF + Cohere Rerank v3 cross-encoder) over ~4,000 financial-document chunks; ~550ms end-to-end.
- Engineered a Modal-split data architecture: text chunks in Qdrant for semantic retrieval; canonicalized GAAP line items in DuckDB (sourced from SEC's XBRL Company Facts API) for SQL-queryable cross-company financial analysis.
- Shipped a full-stack interface: FastAPI backend with Pydantic-validated request schemas and payload-filterable retrieval (ticker, fiscal_year, chunk_type); Next.js 14 + TypeScript + Tailwind split-pane UI with a live citation viewer that renders both narrative passages and HTML financial tables, with click-through to original SEC.gov filings.
- Established eval-first discipline: CI-ready 8-case smoke harness with per-stage score-floor calibration (separate thresholds for cosine, RRF, and rerank-relevance scales), persisted JSON results for regression diffing. Tier-stratified Ragas + custom evaluation harness is the next planned milestone.

Project Kanto: Real-life Pokedex | *Dart, Flutter, Riverpod, Isar (NoSQL), tflite_flutter, Python, PyTorch, Ultralytics YOLOv8, TFLite, ONNX, iNaturalist 2021*.

Apr 2026 - Present

- Built an offline, real-time mobile species classifier across 10,000 species, achieving 38% top-1 / 68% top-5 accuracy with a YOLOv8n model fine-tuned on iNaturalist 2021 Mini and exported to INT8/float TFLite for on-device inference.
- Reduced per-frame ML inference latency $10\times$ ($\approx 2.0s \rightarrow \approx 150ms$) on a mid-range device by rearchitecting the pipeline, eliminating a misaligned detector stage, and switching to zero-copy TransferableTypedData across Dart isolate boundaries.
- Diagnosed a catastrophic preprocessing bug (YOLOv8 emitting normalized coordinates parsed as pixel-space) that had been silently degrading predictions to a single class; built a live debug overlay that surfaced the root cause within minutes.
- Replaced naive vote-based temporal smoothing with logit-space probability averaging across a rolling 12-frame buffer, directly exploiting the classifier's high top-5 / unstable top-1 distribution to lock onto stable ranked candidates.
- Designed and shipped an end-to-end mobile inference pipeline in Dart YUV→RGB decode, 90° sensor-rotation correction, bilinear crop-resize, invoke, softmax smoothing, top-3 HUD) running entirely on a background isolate to keep the UI at 60 fps.
- Authored a full Python ML training pipeline and trained the production model on an A100 (50 epochs, batch 384, imgsz 224).

SmartSubs: Context-Aware Smart Subtitle Engine | *Python, OpenCV, YOLOv8, PySceneDetect*

Feb 2026 - Present

- Engineered an automated video processing pipeline that dynamically calculates optimal subtitle placement across a 6-zone spatial grid to prevent the occlusion of faces and critical action.
- Implemented YOLOv8 object detection with a strict <15% Intersection over Union (IoU) veto threshold to create dynamic "cost heatmaps" across 1080p video frames.
- Designed a temporal aggregation algorithm that samples every 5th frame (achieving an 80% reduction in processing time) to enforce subtitle stability and eliminate visual jitter within continuous scenes.
- Built a conversion engine to parse standard .srt files and output .ass format with precise, frame-accurate coordinate tags, maintaining a 20-pixel safety margin around detected subjects